Event-Anchored Fusion Ancestry and Consumer-Adequate Continuations

Finite Realizations for Single-Defect Circular Actions

WuJun Chen
Independent Researcher
2026

This paper (Paper XXXIX of the RIME program) studies finite representations of event-anchored packet histories and proves complete continuation adequacy for one consumer on a declared singleton-seed-generated domain.

Abstract

An event-anchored consumer of a coalescing packet execution requests a selected fusion's origin snapshot, its complete later absorption sequence, its current carrier, and legal continuation commands. The current placed partition and selected fresh block alone do not determine that origin. We specify the task independently from execution logs and realize it by a finite placed-fusion forest for labelled single-defect circular actions. On singleton-seed-generated domains with $2 \le N \le n$, the decoder recovers the full consumer state from the current packets and the selected node's ancestor path. It reflects command domains and commutes with successors, including unrelated fusion during a carrier-preserving command. Every legal finite continuation therefore runs on the same actual invocation. For a selected fresh block F, the absorption sequence has at most $N - |F|$ records, while execution length is unrestricted. Atom renaming transports the realization. A placed forward union-find-with-explanations (UFE) adapter gives a second realization and a surjective decoder to the generated forest, with matching registration, reports, command domains, and successors. The results establish sufficiency for this consumer on generated forward calls; they assert neither minimality nor an efficient or machine-verified implementation.

Keywords. fusion ancestry; event-anchored consumer; state representation; domain reflection; union-find with explanations

Notation

Symbol	Meaning
$Q = \mathbb{Z}/n\mathbb{Z}$, Ω	positions and source atoms, with $N = \lvert\Omega\rvert$
p, d, $K = \{k_0, k_1\}$, c	labelled cycle, defect, kernel ports, and collision image
h, P_h	chronological execution word and its placed packet endpoint
F, e_F	selected fresh atom block and its unique registered fusion
$\mathcal{A}_F(h)$	complete chronological sequence of later carrier absorptions
$U(h, F)$	independent consumer state, including the full current partition
z_h, J_C	generated forest state and forest-to-consumer decoder
$Y = (u, \ell)$, J_F	placed forward-UFE state and its forest decoder

1 The Log Task and Its Realization

The question is one of state representation and semantic sufficiency: **can a representation omit the complete execution history while preserving a specified fusion event's origin, later causal evolution, and every legal continuation of a declared consumer?** "Every continuation" refers to that consumer's commands on its generated invocation domain, not to arbitrary future queries about the history.

Select an actual binary fusion event in a raw packet execution, identified by its fresh atom block F. The independent task is

$$\mathrm{Track}(h; F) = (\text{origin snapshot}, \mathcal{A}_F(h), \text{current carrier}).$$

The report includes every subsequent fusion that absorbs its carrier, every partner, and the event-time addresses. The selected event need not be the latest event. The consumer also requests `Carry` and `Absorb` commands with precise guards and updates. `Carry` may allow a fusion elsewhere, so its state retains the full current partition rather than only carrier-local output. Section 2 defines the task from execution logs without referring to a forest implementation.

Position and ancestry play different roles. A `p` step preserves the causal absorption sequence but changes the carrier's current position. Event-local addresses remain fixed. Thus an unplaced merge record alone is not the whole requested report. This observation does not imply an expressivity separation from an annotated merge-history implementation.

1.1 Why the current partition is insufficient

A three-state example already separates the current packets from the selected event's origin. Take $Q = \Omega = \mathbb{Z}_3$, keeping position and atom labels conceptually distinct, and let

$$p(q) = q + 1 \pmod 3, \qquad d = (0, 2, 0).$$

The defect has kernel ports 0 and 2 and collision image 0. Start both histories from the same identity-placed singleton seed $P_\epsilon(q) = \{q\}$, and read words chronologically. Compare $h = dd$ with $h' = pdd$. Immediately before the final d, their packets are

History	Packet at port 0	Packet at port 2
dd	$\{0, 2\}$	$\{1\}$
pdd	$\{1, 2\}$	$\{0\}$

Indeed, the first d merges the atoms occupying 0 and 2; the preceding p in the second history changes those atoms to 2 and 1. In both cases the remaining singleton moves to port 2. The final d therefore gives the same placed partition $P_h(0) = P_{h'}(0) = \{0, 1, 2\}$, with no other occupied position. Select the final fusion in each history by its fresh block $F = \Omega$.

Writing $\mathrm{snap}_h(F) = (d, b_h, 0)$ for its origin snapshot, b_h assigns the two parent blocks to their event-time kernel ports. The table gives

$$(P_h, F) = (P_{h'}, F), \qquad \mathrm{snap}_h(F) \neq \mathrm{snap}_{h'}(F).$$

Thus no function of just the current placed partition and selected fresh block can recover the required origin snapshot on this generated domain. The two final invocations even have the same current carrier and empty post-origin absorption sequences. The missing datum is the origin, not the current location or a later absorption.

This is a necessity example for retaining event-history information, not a minimality theorem for the forest. It adds no hypothesis to Theorem 4.1. It also gives no separation from UFE with retained effective-union history: those histories distinguish the two executions. Proposition 5.1 supplies the positive correspondence with that annotated-history representation.

1.2 From specification to continuous realization

Section 2 defines the consumer directly from raw execution logs. Section 3 constructs its finite forest representation, and Section 4 proves complete adequacy and continuous realization. Section 5 gives the placed forward-UFE correspondence. Section 6 follows one selected event through an unrelated fusion and a later absorption. Standard presentation, handoff, and path composition facts are collected in Appendix A. The central results are consumer adequacy and the placed-history representation correspondence.

The proof of Theorem 4.1 follows the chain

$$\begin{gathered}\text{log events} \longleftrightarrow \text{retained nodes}\\ \text{causal absorption sequence} = \text{ancestor path}\\ \text{read coverage} + \text{domain reflection} + \text{successor commutation}\\ \Longrightarrow \text{same-invocation continuous realization.}\end{gathered}$$

The full placed partition is retained because `Carry` can include a fusion elsewhere: both the global packet update and the next collision guard must remain correct. Corollary 4.3 bounds the requested causal data without bounding execution length. Corollary 4.2 follows one actual invocation through its successors, rather than choosing a new existential representative at every step. Proposition 5.1 then shows that an existing effective-union history, with declared placement and port bindings, also realizes the same task.

Related Work and Novelty Boundary

Semantic and causal background

Explicit transition rules and induction over executions are established operational-semantics methods [5]. Simulation and refinement provide a broader framework for comparing implementations with specifications [2]; coalgebra gives a general setting for systems and structure-preserving maps [7]. The partial actions, domain clauses, and continuation induction here do not constitute a new general theory of operational semantics or simulation. Theorem 4.1 supplies those obligations for one concrete model and independently specified consumer.

Finite paths with concatenation and empty identities are standard free-category constructions; see Riehl, Example 4.1.13 [6]. Appendix A uses literal annotated paths and distinguishes their equality from endpoint realization equality. It does not claim priority for path composition or silently identify a handoff step with an empty path.

Nielsen, Plotkin, and Winskel relate nets, event structures, and domains in a theory of concurrency [3]. This is relevant background for separating causal dependence from a total execution chronology. Our forest retains fusion ancestry and declared event-local data; it is not presented as a full event structure with a conflict/concurrency theory. Theorem 4.1 recovers the chronological absorption subsequence along one selected lineage, not the ordering of incomparable events.

Union histories and explanation

Union trees are established objects in disjoint-set algorithms; Gabow and Tarjan study a special case with the union structure known in advance [1]. Nieuwenhuis and Oliveras retain proof forests to support explanations of equivalence in union-find and congruence closure [4]. The nearest comparison is Stevens and Ghidini's verified union-find-with-explanations (UFE) data structure [8]. Their Section 3 retains an ordered list of effective union inputs. Theorem 4.2 proves equality of the simple and refined explanation algorithms; Theorem 5.1 gives an imperative total-correctness specification. These results concern the full verified development, not just one historical query. Our Proposition 5.1 uses the forward abstract interface of Section 3,

together with current placement and an ordered-port call convention. It does not inherit the imperative refinement guarantees.

The task formulated here tracks a selected fusion event under labelled coordinate actions, reports each later carrier absorption and its event-time addresses, and reflects the guards of `Carry` and `Absorb` using the full current partition. It is not a congruence-closure algorithm, an equality-explanation query, or an efficient union-find implementation. We claim no running-time improvement, SMT application, or machine-checked refinement to executable code.

The correspondence supplies registration, full reports, and both continuation commands at the forward abstract ADT boundary. It positively represents the consumer in that annotated-history system, rather than asserting an expressivity separation. Bare-UFE equivalence and rollback are outside the comparison.

Contribution boundary

The contribution is the realization proof for the joint consumer obligation: an event anchor, changing placement, a complete causal absorption sequence, and exact continuation guards. Theorem 4.1 proves these obligations for the finite forest; Proposition 5.1 supplies a second realization through a surjective placed-UFE decoder. The transition framework, simulation method, union-history representation, and path composition are established background. The comparison is selective and establishes no priority or expressivity separation for this combination. Minimality, efficiency, and verified code are separate questions.

2 Execution Logs and the Event-Anchored Consumer

Let Q and Ω denote finite nonempty sets of positions and source atoms, and let A be a finite set of generator labels. A labelled action is a family

$$f_a : Q \longrightarrow Q, \qquad a \in A.$$

Labels remain part of the data even when two transformations coincide. Write $\mathsf{Pkt}(Q, \Omega)$ for placed partitions of Ω: a member P assigns a nonempty block $P(q)$ to each occupied position, the assigned blocks are pairwise disjoint, and their union is Ω. Unoccupied positions have the empty block.

The raw push-forward is

$$(f_{a\#}P)(q') = \bigcup_{q : f_a(q) = q'} P(q).$$

Its rank is the number of nonempty blocks. Atoms are never deleted.

2.1 Labelled executions and registration

The Event-Anchored Absorption Consumer is specified from raw execution logs, before choosing a representation. Fix $Q = \mathbb{Z}/n\mathbb{Z}$, $n \geq 2$, the labelled cycle $p(q) = q + 1$, and a rank-$(n-1)$ defect d with unique binary kernel $K = \{k_0, k_1\}$ and collision image c. Thus $A = \{p, d\}$ with $f_p = p$ and $f_d = d$. Fix $2 \leq N = |\Omega| \leq n$ and an injective singleton seed placement $\iota : \Omega \hookrightarrow Q$.

Words are chronological: uv means run u and then v. A history h is any finite word in these labels, starting from the singleton partition $P_\epsilon(\iota(x)) = \{x\}$. Let P_h be its raw packet endpoint. A fusion occurrence e has snapshot

$$\mathrm{snap}(e) = (d, b_e, c), \qquad b_e(k_i) = \text{packet at } k_i \text{ before } e.$$

Its fresh block is $F_e = b_e(k_0) \cup b_e(k_1)$. A nonempty invocation argument $F \subseteq \Omega$ is registered precisely when a logged event has $F_e = F$. That event e_F is unique: its atoms do not split after creation, and a later fusion of their carrier adds a disjoint nonempty partner.

2.2 Consumer state and report

For a registered F, its current carrier is the unique (q_h, C_h) with $C_h = P_h(q_h) \supseteq F$. The requested absorption sequence $\mathcal{A}_F(h)$ is the chronological subsequence of events after e_F that merge this carrier. Each record contains

$$(d, q_e, C_e, s_e, B_e, c, C_e \cup B_e),$$

where $\{q_e, s_e\} = K$, $C_e = b_e(q_e) \supseteq F$, and $B_e = b_e(s_e)$. Thus it includes both event-time parent addresses and the partner block. The complete consumer state is

$$U(h, F) = (P_h, F, \mathrm{snap}(e_F), \mathcal{A}_F(h)).$$

Its report returns the origin snapshot, the whole absorption sequence, and the current carrier. Only summaries generated by registered histories are admitted. The consumer requests no absolute timestamps or total chronology of incomparable events.

2.3 Commands and summary closure

For a current carrier (q, C) and label a, put

$$G_a(P, q) = \{s \in \operatorname{supp} P : f_a(s) = f_a(q)\}.$$

`Carry(a)` is enabled when $G_a(P, q) = \{q\}$; `Absorb(a)` is enabled when this group has two points $\{q, s\}$. Both commands update **all** packets to $f_{a\#}P$ and keep F and its origin snapshot fixed. `Carry` leaves the absorption sequence unchanged, moving the carrier to $(f_a(q), C)$. `Absorb` necessarily has $a = d$; it appends the displayed record with partner $P(s)$ and gives carrier $(c, C \cup P(s))$. In particular, `Carry(d)` may merge other packets. Both commands require registration; neither is enabled outside that domain. They partition the labelled steps on a registered summary.

These rules are independently determined by log extension. Equal summaries have the same placed partition, selected block, origin, and absorption list. They therefore have the same command domains and the same successor summary. The commands define deterministic partial actions on the generated summary domain, not just on separate histories. Each legal successor is generated by extending the same history, and the selected event remains registered.

3 A Finite Placed-Fusion-Forest Realization

Use the labelled action and atoms of Section 2. The state below records fusion ancestry and current placement without retaining the complete execution word. It is a semantic presentation in the standard sense made explicit in Appendix A. The forest construction itself permits any finite nonempty atom set; the consumer theorem retains Section 2's restriction $2 \le N \le n$ and its injective singleton seed.

3.1 State and equality

A leaf is labelled by one atom. An internal fusion node has two children indexed by the distinct event-time parent positions k_0, k_1, and records the label d and event-time target position c. Each child has a nonempty leaf set, and the two sets are disjoint. Child indexing is by position, not by an arbitrary ordering of a parent list.

A state is $(\mathcal{F}, m)$:

- $\mathcal{F}$ assigns such a tree to each occupied position; all leaves across the forest are distinct and exhaust Ω;
- m is either absent or a particular internal node of this forest, designated as the marked event.

Equality is structural tree equality with literal atom and coordinate labels, equal placements, and equal marked nodes. The leaf set of a node also identifies it uniquely inside a forest: strict descendants have strictly smaller leaf sets, and incomparable nodes have disjoint ones.

Packet projection takes the leaf set of each placed root. The marker is an event reference, not a current position. Its original parent positions remain event data even when the containing tree moves.

This ambient state space consists of well-formed structures. Well-formedness alone does not certify that their recorded events occurred in a specified source run. Such provenance can be established by generation from a declared initial forest. All events in the example below are generated that way.

3.2 Complete update equations

For a letter $a \in \{p, d\}$, group the occupied roots by their destination $f_a(q)$.

If a destination has one incoming root, move that tree there unchanged. If d merges the two occupied kernel positions, create a new root with those two trees, their pre-step positions, and target c as its event data. No other collision is possible for this action.

The marker rule is explicit:

$$m' = \begin{cases} m_{\text{new}}, & \text{if this step creates a fusion node,} \\ m, & \text{otherwise.} \end{cases} \tag{FM}$$

An existing marked node remains a node of the new forest even when it is no longer selected by m'. No event timestamp, plateau-step counter, or complete word prefix is added.

These are the declared model rules. The packet projection alone does not determine the marker update or the retained event data.

3.3 Owned reads

The model owns the following finite, typed read families:

Read	Meaning and domain
Packet, At, Mass, Occupied	leaf sets and current placement
KernelSide	whether a supplied occupied position lies in $\{k_0, k_1\}$
ExactBlock, ContainsBlock, DisjointBlock, UnionAt	extensional queries on explicitly supplied nonempty atom subsets
MarkedFresh	leaf set of m, defined only when a marker is present
MarkedParents	the unordered pair of child leaf sets of m
MarkedEventAddresses	child-position assignment and target stored at m
MarkedContainer	current root position whose tree contains m

No query identifies event-time addresses with MarkedContainer. A marked event need not be a current packet in an arbitrary well-formed state: its node may lie strictly inside a current tree. Undefined marker reads remain undefined when m is absent.

Proposition 3.1 (Well-defined finite realization)**.**

The equations above define total deterministic p, d actions on this state space. They preserve well-formedness, satisfy packet projection compatibility, and give explicit values for every owned read. The state space is finite for fixed Q, Ω, d.

Proof. *Moving roots preserves the partition of leaf atoms. A collision joins exactly two disjoint nonempty trees and hence preserves that partition while replacing two roots by one. The old nodes remain identifiable, and a new marker names the newly created node; otherwise the old marker remains valid. This proves closure and deterministic updating. Taking leaf sets of grouped roots gives precisely the raw union formula.*

Put $N = |\Omega|$. A binary forest with N leaves and r roots has $N - r$ internal nodes, hence $2N - r \leq 2N - 1$ nodes in total. Leaf labels, event labels, event coordinates, root positions, and marker choices all range over finite sets. There are only finitely many such labelled structures. This proves finiteness without bounding word length. Every listed read is a specified function or partial function of these structures.

Plateau cycles can revisit the same state. Fusion ancestry and the declared event-local data are retained, but plateau word history is not. The forest does not explicitly record a total temporal order among incomparable fusion events. Reads requiring such an order need a separate interpretation proof; it cannot be inferred from the ancestry structure alone. The model likewise cannot answer arbitrary queries about the number or order of past plateau steps.

Proposition 3.2 (Atom-renaming handoff)**.**

Every atom bijection $\eta : \Omega \to \Omega'$ induces a strong handoff (as defined in Appendix A) between the corresponding forest presentations, with position map the identity. It renames every leaf, subtree reference, marker, and block argument.

Proof. *The action groups trees by current position, independently of atom names. Renaming therefore commutes with moving roots and with forming the coordinate-indexed fusion node. It also commutes with (FM). Domains of marker reads agree because marker presence agrees. Leaf-set, containment, union, event-address, and container reads obey (R) under the induced maps. The inverse atom bijection supplies the inverse state map.*

If one restricts to states reachable from a particular initial forest, its renamed initial forest must be used on the other side. Arbitrary atom renaming need not preserve one fixed labelled initial seed.

4 Consumer Adequacy and Exact Continuations

4.1 Decoder and generated domain

Let $\mathfrak{F}$ denote the forest presentation of Section 3, with singleton seed z_ι, and set

$$Z_\iota^{\mathrm{reach}} = \{\delta_h^{\mathfrak{F}} z_\iota : h \in \{p, d\}^*\}.$$

This is a closed domain under both generators. It is not the entire ambient space of well-formed forests. A registered forest invocation is $x = (z, F)$ with $z \in Z_\iota^{\mathrm{reach}}$ and an internal node v_F whose leaf set is F. The selected F need not be the latest marker; it is an invocation argument, not a new retained forest field.

Define $J_C x$ using the full packet projection, the local snapshot at v_F, and its strict ancestors from the immediate parent outwards to the current root. At each ancestor, the child containing v_F supplies the carrier parent; the other child supplies the partner. The child indices give their event-time addresses. The placed root supplies the current carrier.

For a command $\alpha = \mathrm{Carry}(a)$ or $\mathrm{Absorb}(a)$, let Δ_α be the independent consumer action. Let $\hat{\delta}_\alpha$ be the forest implementation: on its requested guard, apply $\delta_a^{\mathfrak{F}}$ and keep the same F.

Theorem 4.1 (Complete event-anchored consumer adequacy)**.**

For every declared seed, finite history h, and nonempty $F \subseteq \Omega$, registration is defined in the execution log if and only if its internal event node exists in $z_h = \delta_h^{\mathfrak{F}} z_\iota$. When registered,

$$J_C(z_h, F) = U(h, F),$$

including every report field. For every registered generated invocation x and command α,

$$\hat{\delta}_\alpha(x) \text{ is defined} \iff \Delta_\alpha(J_C x) \text{ is defined.} \tag{CA-Domain}$$

When defined, registration persists and

$$J_C(\hat{\delta}_\alpha(x)) = \Delta_\alpha(J_C x). \tag{CA-Step}$$

Proof. *The singleton seed has no event and no internal node. Every noncollision step moves roots unchanged. Every collision appends exactly one log event and creates exactly one node from the same two kernel roots, with the same child positions, parent leaf sets, label, and target. Old nodes remain present. Induction gives a bijection from logged fusion occurrences to retained internal nodes. Distinct nodes have distinct leaf sets, proving registration equivalence and correctness of the origin snapshot.*

*At creation, v_F is a root and has no later absorption. Thereafter, a step not merging its containing root leaves its ancestor path unchanged, moves that root, and adds no selected absorption. This includes a fusion between two other roots, even if that event replaces the latest marker. A step merging its containing root creates one new outer ancestor and one new absorption record. Every partner, union, label, and event address is supplied by that same event. Induction identifies the complete chronological absorption sequence with the inner-to-outer ancestor path. Packet projection identifies the current carrier and **all** current packets. This proves the complete state and read equality.*

Both command guards now have identical labelled action, placed partition, and current carrier, hence identical collision group $G_a(P, q)$. This proves (CA-Domain). In its singleton case the containing tree and absorption sequence stay unchanged; its position becomes $f_a(q)$, while all other packets still undergo their raw update. In its two-point case the new node appends exactly the required absorption record and gives the stated new carrier. These are precisely the consumer updates, proving (CA-Step). The selected node remains present, supplying registration at the actual successor for the next command.

The decoder is a surjective, domain-reflecting command simulation onto the generated consumer states: every $U(h, F)$ is realized by (z_h, F). No injectivity, minimality, or coverage of another consumer is claimed.

Corollary 4.2 (Same-invocation continuous realization)**.**

Every finite command path starting at $J_C x$ is legal if and only if its forest implementation is legal starting at that same x. At every step, intermediate consumer states and reports agree, with the same selected F. This holds for every generated representative of the decoded consumer state.

Proof. *Apply (CA-Domain) and (CA-Step) successively. Each actual forest successor, not a newly selected representative, realizes the next consumer state. Registration persists, so the induction continues.*

Thus read coverage, domain reflection, and successor commutation provide a sufficient continuous-realization criterion here. This is not a universal minimal-quotient theorem.

Corollary 4.3 (Bounded causal data for unbounded histories)**.**

If the absorption sequence has t records with successive partner blocks $B_1, \ldots, B_t$, then

$$C_h = F \sqcup B_1 \sqcup \cdots \sqcup B_t, \qquad t \leq N - |F|. \tag{CA-Bound}$$

Proof. *At the origin event the carrier is F.* `Carry` *leaves its atom set unchanged. Each* `Absorb` *adds a nonempty partner disjoint from the entire current carrier. Consequently the partners are also mutually disjoint and never leave the carrier. Induction proves the disjoint-union formula, and*

$$|F| + \sum_{i=1}^{t} |B_i| = |C_h| \le N, \qquad t \le \sum_{i=1}^{t} |B_i| \le N - |F|.$$

No execution-length bound or finite replay is used.

Every fusion reduces the total root count by one, so the whole history has at most $N-1$ fusions at fixed N, including unrelated ones. Execution histories can nevertheless be arbitrarily long because plateau steps need not create events. The consumer's requested causal event data are bounded; all its value fields range over finite sets, giving a finite generated consumer state space. This agrees with the forest bound $2N - r \le 2N - 1$, without adding a counter. Neither bound is a word-length or credit-settlement claim.

Corollary 4.4 (Atom-renaming transport)**.**

An atom bijection η transports registration, both command domains, reports, and continuations, using seed $\iota' = \iota\eta^{-1}$ and selected argument $\eta(F)$. Atom blocks are renamed; generator labels and event-time positions stay fixed.

Proof. *Proposition 3.2 transports the selected node, its ancestor path, and the whole packet partition. It therefore transports J_C, its collision guards, and (CA-Step); Corollary 4.2 transports continuations.*

Theorem 4.1 and its corollaries establish sufficiency for the independently specified consumer. Section 5 supplies another realization of the same task without enlarging its read or command signature.

5 A Placed Forward-UFE Correspondence

Use the singleton initialization, equivalence-class operations, ordered effective-union history, and effective union of the abstract UFE interface [8]. Fix an enumeration of the existing atoms. A comparison state $Y = (u, \ell)$ adds only an injective placement ℓ of all current UFE classes in Q. It stores no fusion forest or consumer report.

At a raw fusion, submit one atom from the packet at k_0 as the first union argument and one from the packet at k_1 as the second; use a fixed choice by atom index. The update merges those classes and moves all classes by the raw action. A nonfusion only updates placement. Restrict to states generated by these updates from the same singleton seed. This is a comparison adapter, not an alteration of the forest or consumer specification.

Define $J_F(Y)$ by folding the retained ordered union list into binary event nodes and placing its roots using ℓ, starting with singleton leaves. Each union replaces its two parent trees by the kernel-port-indexed node; the latest node is marked, with no marker when the list is empty. Prefix equivalence classes recover the two parents of every effective union; the ordered call contract assigns them to the two fixed kernel ports. The same prefix data define $K_U(Y, F)$ directly as the consumer summary, without storing or consulting a forest during the update. For invocation maps put $\widetilde{J}_F(Y, F) = (J_F(Y), F)$.

Proposition 5.1 (Placed forward-UFE correspondence)**.**

On the declared generated comparison domain:

1. *$J_F(Y_h) = z_h$ for every raw history h, and J_F is surjective onto the seed-generated forest domain.*

2. *Registration agrees in both representations. When registered,* $K_U(Y,F) = J_C(\widetilde{J}_F(Y,F)) = U(h,F)$ *for a history generating* Y*.*

3. *For each requested* `Carry` *or* `Absorb` *command, the adapter and forest invocation domains agree in both directions. When defined,*

$$\widetilde{J}_F(\Delta^U_\alpha(Y,F)) = \hat{\delta}_\alpha(\widetilde{J}_F(Y,F)). \qquad \text{(UFE-Step)}$$

Their decoded consumer successors and all continuous reports therefore agree on the same actual invocation.

4. *Atom-renaming transport holds when the enumeration, seed, placement, and selected argument are transported together.*

Proof. At initialization both partitions are singleton. A nonfusion leaves the UFE history and forest trees unchanged and moves all current classes/roots. A fusion submits atoms in two distinct current classes, so the UFE call is effective. Their prefix classes are exactly the two raw parents. Appending that ordered union creates exactly the decoder node with the same kernel-port indexing, label, target, and latest marker as the forest update. Induction gives clause 1; taking the same history proves surjectivity.

The resulting fresh blocks are unique event identifiers. Reading the prefix classes after the selected origin selects precisely the later unions whose parent contains F, with every required snapshot field. Current class placement supplies the full partition and carrier. This proves clause 2. Both command guards consequently compute the same collision group. For `Carry(d)` with an unrelated fusion, the adapter still performs that effective union and updates every class, without appending a selected absorption. An absorption adds precisely the selected new record. In both cases F persists and the actual successor commutes, proving clause 3 and its path consequence. Under transported enumeration, the numeric UFE inputs stay the same while decoded blocks rename, proving clause 4.

This is a surjective forward representation and command simulation, not a state isomorphism or full UFE API equivalence. The adapter has current placement and a proved ordered-port call policy; arbitrary UFE histories do not thereby acquire raw-event provenance. Efficient/imperative refinement, rollback, and equality-explanation queries are outside this result.

6 One Event Through Unrelated Fusion and Later Absorption

Take $\Omega = Q = \mathbb{Z}_7$ as labelled sets, keeping atom labels and positions distinct, and

$$p(q) = q + 1, \qquad d = (0,3,2,5,1,4,0).$$

The kernel ports are 0 and 6 and the collision image is 0. Start with singleton trees at their matching positions and no marker. The chronological word *dppd* first creates the event on $\{0,6\}$, then the event on $F = \{3,5\}$. The resulting state z_0 has packet projection

$$0 : \{3,5\}, \quad 1 : \{2\}, \quad 2 : \{0,6\}, \quad 4 : \{1\}, \quad 5 : \{4\}.$$

Register the latter event once, with invocation argument $F = \{3,5\}$. Its origin has parents $\{3\}$ at port 0 and $\{5\}$ at port 6; its absorption list is empty. The latest marker initially selects this event.

6.1 Carry need not mean that no fusion occurs

Run $u = ppd$. The support trace is

$$\{0,1,2,4,5\} \to \{1,2,3,5,6\} \to \{0,2,3,4,6\} \to \{0,1,2,5\}.$$

The two rotations move the selected carrier to positions 1 and 2. At the terminal defect it is at position 2, away from the kernel. That step fuses $\{4\}$ at port 0 with $\{1\}$ at port 6, giving

$$\pi(z_3): \quad 0:\{1,4\}, \quad 1:\{0,6\}, \quad 2:\{3,5\}, \quad 5:\{2\}.$$

All three consumer commands are therefore Carry commands, including the final Carry(d). The latest marker becomes $\{1,4\}$, but the invocation still selects $F = \{3,5\}$, with its original snapshot and empty absorption list. The full partition changes even though the selected ancestry does not.

6.2 The same invocation records the later absorption

From z_3, run $v = p^5 d$. Immediately before its final defect the packets are

$$0:\{3,5\}, \quad 3:\{2\}, \quad 5:\{1,4\}, \quad 6:\{0,6\}.$$

The five rotations are Carry(p) steps. The final Absorb(d) merges the selected carrier at 0 with its partner at 6 and appends exactly

$$(d, 0, \{3,5\}, 6, \{0,6\}, 0, \{0,3,5,6\}).$$

The resulting partition is

$$0:\{0,3,5,6\}, \quad 4:\{1,4\}, \quad 5:\{2\}.$$

The selected origin has not changed. The registered event is now an internal node of its carrier tree; the newly marked event is the larger fusion. Thus one actual invocation exhibits an unrelated fusion followed by a selected absorption. It is not reconstructed from a fresh event choice at each endpoint.

6.3 Atom renaming transports the entire invocation

Insert $\eta = (1\ 2)$ after u, renaming the entire forest and the invocation argument. Here $\eta(F) = F$. Running the same v gives

$$0:\{0,3,5,6\}, \quad 4:\{2,4\}, \quad 5:\{1\}.$$

Proposition 3.2 and Corollary 4.4 transport all event data and consumer reports. In particular,

$$\delta_v(h_\eta(\delta_u z_0)) = h_\eta(\delta_{uv} z_0).$$

This is an equality of full forest states under the declared atom renaming, not merely of masses. Both words have one strict drop, but neither shortestness nor a separate checkpoint-admission predicate is asserted.

7 Computational Artifacts

The paper-owned development package is under `experiments/paper39/` in the RIME repository. Paths in the table are relative to that directory.

Surface	Location	Role
Entry	`README.md`	fixed scope and replay commands
Replay	`consumer_audit.py`, `results/`	bounded three-model comparison
Lean	`lean/`	partial spine and coverage manifest
Validation	`validation/`	local replay and contract tests
Closure	`development-manifest.json`	selected development source bytes

The bounded three-model replay compares the independently written occurrence-log consumer, placed forest, and abstract placed forward-UFE adapter on the fixed finite catalogue. It checks registration, complete reads, command domains, actual successors, and the worked examples. The validator reruns the same audit runner: its status is `LOCAL_CLOSURE_VERIFICATION`, not independent validation or a proof for arbitrary histories.

The partial Lean spine checks fifteen declarations about port-indexed lookup, binary-forest counts, and conditional same-representative path lifting. The path lemmas assume the single-step commutation that the concrete adequacy theorem must supply. The formalization does not prove the full forest/log adequacy or placed forward-UFE correspondence, and does not verify the Python implementations. Default package validation checks the formal source closure; the optional `--lean` mode performs an isolated compiler and axiom replay.

Neither the bounded replay nor the partial Lean spine is a machine proof of Theorem 4.1 or Proposition 5.1. Their data-independent proofs are in this manuscript. The development closure is not a release identity.

8 Claim Status and Boundary

Item	Mathematical scope
Deterministic replay and projection	standard background in Appendix A
Strong-handoff word transport and path composition	standard background in Appendix A
Placed-fusion-forest realization and finiteness	explicit construction and Proposition 3.1
One invocation through unrelated fusion, absorption, and atom renaming	worked exact illustration of the declared equations
Complete consumer adequacy and continuous realization	Theorem 4.1 and Corollary 4.2; independent log specification, closed seed-generated domain
Bounded absorption chain and finite consumer data	Corollary 4.3; derived from atom conservation and disjoint partner addition, not finite replay
Consumer atom-renaming transport	Corollary 4.4; seed and selected argument transported together
Placed forward-UFE representation and commands	Proposition 5.1; abstract effective-union history plus current placement, no state isomorphism or imperative-wrapper verification
Bounded three-model replay and hostile controls	local consistency checks on the fixed finite catalogue; not general theorem proofs or independent validation
Partial Lean implementation spine	port-lookup invariance, forest-count identities, and conditional path lifting only; not concrete consumer adequacy or UFE correspondence
Current partition and selected fresh block determine the origin	false on the three-state example in Section 1.1
Minimal representation, efficient implementation, or full UFE API equivalence	not established

Conclusion

The answer to the opening question is affirmative on the declared singleton-seed-generated domain. The forest omits the complete execution word and the total chronology of incomparable events while retaining the selected origin, its entire later absorption sequence, and the full current partition needed by the command guards. Theorem 4.1 proves those reads, domains, and updates correct. Corollary 4.2 keeps the same actual invocation through every legal finite continuation; it is stronger than separate one-step existence claims.

The three-state example explains why current packets alone cannot replace this historical information. Proposition 5.1 explains why the forest is not the only representation that can supply it: effective-union history, with placement and ordered-port binding, realizes the same consumer. Together these results delimit a state-sufficiency and representation study, not an expressivity hierarchy of data structures.

A future reduction of the retained state must preserve registration, the declared reads, command domains, and successors on its continuation domain. The results here establish those obligations for two representations, while leaving minimality, efficiency, and compatibility with other typed interfaces to separate arguments.

Appendix A Semantic Presentations, Handoffs, and Paths

This appendix records standard semantic and composition facts used by the realizations. It does not introduce another consumer or an admission rule. Here Q, Ω, and A may be arbitrary finite nonempty position, atom, and label sets, with declared maps $f_a : Q \to Q$. Use the placed

partitions and raw push-forward construction of Section 2 for these data. The main realizations specialize this background to the declared circular action.

A.1 Semantic presentations

Definition Appendix A.1 (Semantic presentation)**.**
A presentation $\mathfrak{S}$ consists of the labelled action above and:

$$\left(Z, \pi, (\delta_a)_{a\in A}, (B_o, D_o, V_o, \mathrm{read}_o)_{o\in O}\right).$$

Here Z is a set with declared equality, $\pi : Z \to \mathsf{Pkt}(Q, \Omega)$ is packet projection, and $\delta_a : Z \rightharpoonup Z$ is a partial deterministic action. The finite read signature O declares argument types B_o, domains $D_o \subseteq Z \times B_o$, value types V_o, and maps $\mathrm{read}_o : D_o \to V_o$. For an unparameterized read take B_o to be a singleton. Domains may therefore depend on both the state and the supplied arguments; reads are not untyped lookups.

The following requirements are explicit:

1. **Projection compatibility.** Whenever $\delta_a(z)$ is defined, $\pi(\delta_a(z)) = f_{a\#}\pi(z)$.
2. **State-local definedness.** The domain and value of δ_a depend on z and the declared presentation, not on an unrecorded execution history or a desired downstream answer.
3. **Declared read coverage.** The presentation lists the reads it owns and their domains. A consumer requiring further reads has not yet been interpreted by this presentation.
4. **Separate admissibility.** No checkpoint, lower-success, or projectability predicate is obtained merely by giving a state in Z.

These data must specify the state, equality, domains, and update equations; "all information needed later" is not such a specification. Read coverage concerns the declared signature, not all possible external consumers. Put $r(z) = \operatorname{rank} \pi(z)$. Equal packet projections need not mean equal states; undefined reads and transitions receive no defaults. Finiteness of Z requires its own proof and does not follow from finite Q.

A.2 Replay and first exits

Words are chronological: uv means run u and then v. Define $\delta_\varepsilon(z) = z$ and $\delta_{wa}(z) = \delta_a(\delta_w(z))$ when both operations are defined.

Proposition Appendix A.2 (Deterministic replay and projection)**.**
For a fixed presentation, source z, and word w, there is at most one legal state trace. If it exists, its packet projection is raw replay of w.

Proof. *The empty trace is fixed by z. Each next state is the value of the declared partial function δ_a at the preceding state, hence is unique if defined. Projection compatibility at each step proves the second assertion by induction.*

Existence is not implied by uniqueness. A raw word can be defined while its typed replay is undefined.

*A **replay path** is the literal word together with its typed source and presentation; its trace is the unique one above. Endpoint equality does not identify two words, two traces, or two certificates about those traces.*

*A **first exit of unit drop** from rank r is a nonempty replay path all of whose proper prefix states have rank r, with final rank $r-1$. An **elementary transfer path** has the form ev, where e is such a first exit and v stays at rank $r-1$. The empty tail is allowed.*

*A **general transfer path** is any finite legal replay path; its strict-drop events remain in its trace. Its empty path is an identity and has no strict drop. Two elementary paths with successive unit drops concatenate to a general path with two drops, not to one elementary transfer path.*

These definitions do not select endpoint-shortest representatives. A shortest-path filter, simple-path authorization, checkpoint rule, or accounting condition is a further predicate on paths. Its preservation under concatenation requires a separate argument.

A.3 Strong handoffs

A handoff changes a semantic presentation or its coordinates; it is not another generator action. We first give a strong sufficient notion. It is not asserted to describe every historical boundary handoff.

Definition Appendix A.3 (Strong labelled handoff)**.**

For presentations $\mathfrak{S}, \mathfrak{S}'$ with the same labelled alphabet, a strong handoff h consists of bijections

$$h_Q : Q \to Q', \qquad h_\Omega : \Omega \to \Omega', \qquad h_Z : Z \to Z',$$

and declared bijections on the corresponding read-value and argument sorts. Truth values and numerical semantic values are fixed. Maps on atom subsets and coordinate-indexed arguments are induced by h_Ω and h_Q; other sort maps must respect their declared structure. Read names are matched explicitly. Write $h_\# P$ for the placed partition with $(h_\# P)(h_Q(q)) = h_\Omega(P(q))$.

For each label a, state z, and declared read o, require:

$$\begin{aligned} h_Q f_a &= f'_a h_Q, \\ \pi'(h_Z z) &= h_\# \pi(z), \\ z \in \operatorname{dom} \delta_a &\iff h_Z z \in \operatorname{dom} \delta'_a, \\ h_Z(\delta_a z) &= \delta'_a(h_Z z) \quad \text{when defined.} \end{aligned} \tag{H}$$

Read domains satisfy $(z, b) \in D_o \iff (h_Z z, h_{\mathrm{arg}} b) \in D'_o$; on those domains,

$$\mathrm{read}'_o(h_Z z; h_{\mathrm{arg}} b) = h_o(\mathrm{read}_o(z; b)). \tag{R}$$

For an unparameterized read omit b. Handoff equality means equality of these maps, not merely agreement at one observed endpoint.

Present-read agreement alone does not imply (H). A transfer supplies its particular handoff; existence of some handoff does not make one canonical.

Proposition Appendix A.4 (Word transport)**.**

For every word w, $\delta_w(z)$ is defined if and only if $\delta'_w(h_Z z)$ is defined; when defined,

$$h_Z(\delta_w z) = \delta'_w(h_Z z). \tag{WT}$$

Proof. *The empty word uses the identity map. For the induction step, the induction hypothesis matches the prefix states, and (H) matches the domain and value of the next action.*

Identities, componentwise composites, and inverses satisfy (H) and (R), using the biconditional domains for inverses. These are the usual strong-handoff operations; endpoint-read preservation alone does not supply them.

A.4 Literal path composition

An exact **boundary-transfer witness** can be displayed as

$$\tau = (\mathfrak{S}, z^-, w, z^+;\ h, \mathfrak{S}', \hat{z}^+), \qquad z^+ = \delta_w z^-, \quad \hat{z}^+ = h_Z z^+. \tag{T}$$

It supplies a legal replay and a particular strong handoff, with source $(\mathfrak{S}, z^-)$ and target $(\mathfrak{S}', \hat{z}^+)$. Its stored endpoint fields must satisfy (T); they are not independent assertions. The replay may additionally be qualified as elementary. This is a witness type for these declared presentations, not a certificate for a separately supplied checkpoint or admissibility relation.

Choose a collection of presentations and strong handoffs closed under identities and composition. An object is a pair $(\mathfrak{S}, z)$. A **handoff-annotated transfer path** is a finite sequence of:

- legal generator steps inside one presentation;
- declared handoff steps $(\mathfrak{S}, z) \to (\mathfrak{S}', h_Z z)$.

Adjacent endpoints must be literally equal as typed objects. A handoff matching two unequal endpoints must be supplied as a step, not silently inserted by matching their masses.

Expanding (T) gives generator steps followed by its handoff. Composites retain intermediate handoffs and need not have one-word/one-handoff form. The endpoint realization in Proposition Appendix A.6 does not identify expanded witness histories.

Proposition Appendix A.5 (Literal path composition)**.**

Concatenation of matching annotated paths is defined, associative, and has the empty sequence at each object as a two-sided identity.

Proof. *Each step is already legal; matching endpoints supply the only new adjacency. Both bracketings of three paths yield the same ordered step sequence, and an empty sequence changes none of its entries.*

A nonempty identity-handoff step is not the empty path, and two handoff steps are not literally one composite-labelled step.

Proposition Appendix A.6 (Replay across one handoff)**.**

Suppose u is legal from z in $\mathfrak{S}$, $h : \mathfrak{S} \to \mathfrak{S}'$ is strong, and v is legal from $h_Z(\delta_u z)$ in $\mathfrak{S}'$. Then uv is legal from z in $\mathfrak{S}$ and

$$\delta'_v(h_Z(\delta_u z)) = h_Z(\delta_{uv} z). \tag{COMP}$$

Proof. *Apply Proposition Appendix A.4, including its definedness reflection, to v at $\delta_u z$. Chronological word evaluation then gives (COMP).*

Thus $u; h; v$ and $uv; h$ have equal typed final objects, not equal literal paths or validation records. A further path quotient requires its own congruence.

Generator length and rank-drop count are additive under literal path concatenation; handoffs contribute zero to both. No maturity, surplus, debt, budget, or shortest-fiber closure follows without its own definition and compatibility theorem.

Checkpoint certification is separate. One may supply a relation $\mathrm{Check}_{\mathfrak{S}}(z, \beta)$ with witness β. A legal path ending at z does not produce such a witness, and a strong handoff does not transport it unless that certification relation has its own preservation law. This prevents semantic composability from becoming recursive admission by definition.

A.5 Negative controls

A.5.1 Present agreement does not preserve enabled continuations

Let $Q = \{0, 1\}$, let p swap the positions, and let d map both to 0. Take states (P, b) with $b \in \{0, 1\}$, packet projection P, and reads restricted to packet data. Define p everywhere with b unchanged; define d only when $b = 0$, again leaving b unchanged.

The map $(P, b) \mapsto (P, 1-b)$ preserves every declared packet read but not the domain of d. It is not a strong handoff. This explicit alternative presentation shows why domain compatibility in (H) cannot be replaced by equality of endpoint observations.

A.5.2 Finite positions do not force finite typed states

Adjoin a natural-number counter to placed partitions and increment it on each generator step. Projection compatibility still holds. Repeated p^n returns the packet placement but increases the counter, producing infinitely many typed states. Thus finite positions and a finite raw action do not justify a finite typed graph. Proposition 3.1 used a genuine bounded-node argument for its particular retained state.

A.5.3 Shortest representatives need not compose to a shortest one

Take a presentation with three typed states s, m, t projecting to the same rank-one packet placement. With raw identity actions, define only the typed edges $s \xrightarrow{a} m$, $m \xrightarrow{b} t$, and $s \xrightarrow{c} t$. Each of the first two edges is shortest between its endpoints, but ab is not shortest from s to t.

This example satisfies the partial-action requirements. It separates the path composition rule from any endpoint-shortest subrelation; it does not assert closure of an externally specified shortest-path relation.

A.5.4 An identity path is not a strict transfer

The empty path has no rank drop. A composite of two successive elementary transfers has two. Neither belongs to the exactly-one-drop elementary class. A category claim for that class with ordinary path composition would therefore require changing its objects or arrows; it cannot follow from calling its records typed.

Appendix B Supplementary Proof Notes

The expanded specification and proof notes are available in the RIME repository, under `papers/paper39/`. The manuscript states and proves the central results; these notes provide additional definitions, case analysis, and source locators.

Supplement	Short path
Independent log specification	`ABSORPTION_CONSUMER_SPEC.md`
Event/node and absorption proof	`ABSORPTION_CONSUMER_ADEQUACY.md`
Ordered UFE calls and decoder	`UFE_CORRESPONDENCE.md`

These short paths and the repository landing link are source locators, not immutable release pins. The development manifest in Section 7 binds the selected manuscript and supplementary source bytes.

The examples in Sections 1.1 and 6 are worked applications of the equations. Section 7 distinguishes their bounded replay from the partial formalization; neither replaces the manuscript's general proofs.

References

[1] Harold N. Gabow and Robert Endre Tarjan. A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences*, 30(2):209–221, 1985.

[2] Nancy Lynch and Frits Vaandrager. Forward and backward simulations, part I: Untimed systems. *Information and Computation*, 121(2):214–233, 1995. Author-hosted technical-memo version; journal metadata from the author's publication list.

[3] Mogens Nielsen, Gordon Plotkin, and Glynn Winskel. Petri nets, event structures and domains, part I. *Theoretical Computer Science*, 13(1):85–108, 1981.

[4] Robert Nieuwenhuis and Albert Oliveras. Proof-producing congruence closure. In *Term Rewriting and Applications*, volume 3467 of *Lecture Notes in Computer Science*, pages 453–468. Springer, 2005.

[5] Gordon D. Plotkin. A structural approach to operational semantics. *The Journal of Logic and Algebraic Programming*, 60–61:17–139, 2004. Reprint of the 1981 Aarhus notes, DAIMI FN-19.

[6] Emily Riehl. *Category Theory in Context*. Dover Publications, 2016.

[7] J. J. M. M. Rutten. Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000.

[8] Lukas Stevens and Rebecca Ghidini. Simplified and verified: A second look at a proof-producing union-find algorithm. In Clark Barrett and Uwe Waldmann, editors, *Automated Deduction – CADE 30*, volume 15943 of *Lecture Notes in Computer Science*, pages 261–279. Springer, 2025. The proposition comparison uses arXiv version 2, dated 11 August 2025.